NTNU
Norwegian University of
Science and Technology

Practical Anonymous Communication with Homomorphic Encryption

Angelo, Imre

Submission date: May 2026
Main supervisor: Associate Professor Park, Jeongeun, NTNU

Norwegian University of Science and Technology
Department of Information Security and Communication Technology

Problem description

Many anonymous communication systems have been proposed and implemented with the goal of allowing users to exchange messages over a network without revealing who is communicating with whom. Most of these designs rely on the *threshold model* to provide anonymity, wherein some components of their infrastructure must be behaving honestly. Systems with stronger anonymity guarantees generally suffer from poor scalability or are impractical to instantiate.

This thesis examines a new construction called a (Somewhat) Practical Anonymous Router (sPAR), which addresses the trust and practicality concerns of existing systems. The construction relies only on well-established Fully Homomorphic Encryption (FHE) schemes, making it realistically implementable. While sPAR presents a promising new avenue for constructing anonymous communication systems, it remains a theoretical construction. A concrete implementation and a deeper investigation is therefore needed to evaluate its practical performance and viability in larger networks.

In an effort to investigate sPAR's viability, this thesis aims to implement the scheme and conduct a structured evaluation of its performance with respect to the number of participants in the system.

Research Questions:

- What is the computational cost of sPAR, and how does it compare to the theoretical complexity described in the original paper?
- How does the performance of sPAR scale with the number of participants, and at what point does it become impractical?

Approved: 2026-04-01 – Associate Professor Park, Jeongeun, NTNU (Main supervisor)

Abstract

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

This is the second paragraph. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

And after the second paragraph follows the third paragraph. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Sammendrag

After this fourth paragraph, we start a new paragraph sequence. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

This is the second paragraph. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Preface

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Contents

List of Figures ix

List of Tables xi

List of Algorithms xiii

List of Symbols xv

List of Acronyms xix

1 Background 1

2 Related Works 3

2.1 Gadget Decomposition in RNS . . . 3
2.1.1 Brakerski-Vaikuntanathan . . . 3
2.2 Implementation Details . . . 5
2.2.1 Residue Number System . . . 5
2.2.2 Gadget Decomposition . . . 5
2.2.3 Key Switching . . . 6
2.2.4 RNS Gadget . . . 6
2.3 TFHE (RGSW) . . . 8
2.3.1 Parameter Choice . . . 9
2.3.2 External Product (textbook) . . . 9
2.3.3 Internal Product (textbook) . . . 9
2.3.4 The Hybrid-Keyswitch RNS Gadget . . . 11
2.3.5 RLWE-to-RGSW Expansion from ORAM . . . 16
2.3.6 Homomorphic Field Trace . . . 17
2.4 Implementation Details . . . 18
2.4.1 RGSW Encryption (textbook form) . . . 18
2.4.2 V2 RGSW Encryption . . . 19
2.5 (Somewhat) Practical Anonymous Routing . . . 19
2.5.1 HomPlacing . . . 20
2.6 Multi-Party Homomorphic Encryption . . . 23

3 Methodology **25**
3.1 Variables 25
3.2 Notes 25
3.3 Residue Number System 25

4 Results and Discussion **27**

5 Conclusion and Future Work **29**

References **31**

Appendices

A Algebra **33**
A.1 Rings 33
A.2 Cyclotomic Polynomials 33
A.3 Galois Group of Cyclotomic Polynomials 35

B Number Theory **37**
B.1 Chinese Remainder Theorem 37
B.2 CRT Polynomial 37
B.3 Double CRT Polynomial 37
B.4 Number Theoretic Transform 37

List of Figures

2.1 HomPlacing tree with a 3-bit index a. At level i, the i-th MSB of the index a determines homomorphically wether the ciphertext is placed in the left (0) or right (1) bucket. This process is repeated $\log n = 3$ times until the ciphertext is placed in the a-th bucket, without the placer ever knowing which bucket that is. The highlighted path shows the route taken for the index $a = 6 = [110]_2$: the MSB places the ciphertext in the right bucket, and so on. Again, the server doesn't know a, only the *encryption* of the bits $[a]_2$. 20

A.1 TODO: Create in tikz or drop 34

List of Tables

2.1 V1 (textbook) vs V2 (hybrid-keyswitch) RGSW. The V2 form reduces the inner loop length by $\ell/\mathsf{dnum} \approx k/2$, replaces ciphertext-level operations with `DCRTPoly`-level operations, and reuses OpenFHE's hybrid-keyswitch machinery for digit decomposition and modulus reduction. 15

List of Algorithms

2.1 Client-side RGSW generation 9
2.2 V2 Ring-GSW (RGSW) encryption (server-side, mirrors hybrid keyswitch) 13
2.3 HomExpand 16
2.4 expandRlwe 17
2.5 expandRlwe 17
2.6 Homomorphic placing HomPlacing for one slot 21

List of Symbols

α	Number of RNS limbs per Q-partition: $\alpha = \lceil k/\mathsf{dnum} \rceil$.
dnum	Number of Q-partitions in the hybrid-keyswitch gadget; gadget length in V2 RGSW.
$\boxdot$	External product.
g_j	Partition-indicator gadget element in R_{QP}: value $[P]_{q_i}$ on Q-towers in partition j, 0 elsewhere.
$\boxtimes$	Internal product (RGSW $\times$ RGSW).
ℓ	Homomorphic multiplication level (textbook gadget length).
P	Auxiliary product of primes used to extend $Q \to QP$ for hybrid keyswitching.

List of Acronyms

CRT Chinese Remainder Theorem.

DC-net Dining Cryptographers network.

FHE Fully Homomorphic Encryption.

GSW Gentry-Sahai-Waters.

HE Homomorphic Encryption.

LWE Learning With Errors.

MCFE Multi-Client Functional Encryption.

MPC Multi-Party Communication.

NIAR Non-Interactive Anonymous Router.

NTNU Norwegian University of Science and Technology.

NTT Number Theoretic Transform.

RGSW Ring-GSW.

RLWE Ring Learning With Errors.

RNS Residue Number System.

SHE Somewhat Homomorphic Encryption.

sPAR (Somewhat) Practical Anonymous Router.

Chapter 1
Background

Anonymous communication aims to enable users to exchange messages over a network without revealing who is communicating with whom, thereby protecting both sender and receiver identities. Many such systems have been proposed and implemented over the years, such as Tor, I2P, and mixnets. Most of these designs depend on the *threshold model* to provide anonymity, requiring at least some components of their infrastructure to behave honestly. In practice, this assumption is increasingly strained by large-scale surveillance, correlation attacks, and the growing centralization of internet infrastructure [SEV+15].

Systems with stronger anonymity guarantees based on Multi-Party Communication (MPC) exist. However, these suffer from poor scalability due to their high interactivity, making them impractical for large-scale networks. A Dining Cryptographers network (DC-net), for example, can provide anonymity even if there are no trusted components in the network infrastructure. Doing so requires every participant to interact with every other participant, for every round of messages sent.

Recently, a new line of research has proposed fundamentally different designs, aiming to offer strong anonymity even in the presence of compromised or adversarial components in the network actively attempting to break anonymity. The first such design was the Non-Interactive Anonymous Router (NIAR) [SW21]. This paper proposed using Multi-Client Functional Encryption (MCFE) to allow a single, untrusted router to shuffle or permute messages from a set of clients *obliviously*; without learning anything about the permutation. Thereby making it infeasible for the router to link any of the messages to its sender.

NIAR was a breakthrough in anonymous communication, not only showing the theoretical possibility of designing anonymous routers, but also that such routers can be *non-interactive*: clients only send one message to the router and do not need to perform any additional interaction to achieve anonymity. Using a centralized (anonymous) router in this way yields similar benefits to existing MPC-based systems,

without needing any interactions between clients. However, there are a number of problems that makes NIAR impractical, if not impossible, to implement in practice.

1. The cryptographic components required by NIAR are extremely computationally expensive or not known to be implementable in practice.
2. Achieving anonymity for recipients assumes the existence of a very strong primitive called *indistinguishability obfuscation with sub-exponential security*. The practicality of such a construction is unknown.
3. Reusing the same setup across multiple rounds makes messages from the same sender linkable, unless the expensive setup phase is rerun for every round

- Anonymous communication
- Current solutions
- Problems with current solutions
- FHE as a new approach
- sPAR
- RSA is multiplicatively homomorphic, but (follow-up) paper theorized a *fully* homomorphic scheme may be possible [Gen09; RAD78]

Chapter 2
Related Works

This section details RGSW (External/Internal products), HomExpand, and the sPAR protocol itself.

2.1 Gadget Decomposition in RNS

Residue Number System (RNS) does not support all operations natively, this includes gadget decomposition.

2.1.1 Brakerski-Vaikuntanathan

Standard digit decomposition with $\ell = \lfloor \log_\omega Q \rceil + 1$

$$D_{\omega,Q}(a) = a \cdot \overbrace{\left([1]_\omega , \left[\left\lfloor \frac{1}{\omega} \right\rceil \right]_\omega , \cdots , \left[\left\lfloor \frac{1}{\omega^{-\ell}} \right\rceil \right]_\omega \right)}^{\text{gadget vector } g}$$
$$P_{\omega,Q}(a) = \left([a]_Q , [a\omega]_Q , \cdots , \left[a\omega^\ell\right]_Q \right)$$

Then one row/comp of RGSW is given by (2.1) and the external product by (2.2).

$$C_i = Z_i + D_{\omega,Q}(\mu)_i \tag{2.1}$$
$$x \boxdot Y = \langle P_{\omega,Q}(x), Y \rangle \tag{2.2}$$

RNS

The BV technique can be adapted to RNS by decomposing the values according to each residue as done in [BEHZ16; KPZ21]. Let $Q_i = q_0 q_1 \cdots q_i$ where i is ????

$$D_{Q_i}(a) = \left(\left[a \left(\frac{Q_i}{q_0} \right)^{-1} \right]_{q_0}, \left[a \left(\frac{Q_i}{q_1} \right)^{-1} \right]_{q_1}, \cdots, \left[a \left(\frac{Q_i}{q_i} \right)^{-1} \right]_{q_i} \right)$$

$$P_{Q_i}(a) = \left(\left[a \left(\frac{Q_i}{q_0} \right) \right]_{Q_i}, \left[a \left(\frac{Q_i}{q_1} \right) \right]_{Q_i}, \cdots, \left[a \left(\frac{Q_i}{q_i} \right) \right]_{Q_i} \right)$$

Note that $[a]_B \to [-B/2, B/2)$ is the signed representation.

2.2 Implementation Details

2.2.1 Residue Number System

- In BGV, BFV, CKKS, the ciphertext modulus q is large, much larger than a computer word (typically 64 bits on modern computers)
- Multi-precision arithmetic is know to be slow due to its serial nature
- A RNS represents numbers as a set of smaller residues
- Using CRT, we can get numbers that all fit within a register/word

Definition 2.1. Coefficient Format Given a (canonical) polynomial $a = \sum_{i=0}^{N-1} c_i X^i$ in the ring $R_q = \mathbb{Z}_q[X]/(X^N + 1)$, that is with (up to) N coefficients modulus $q = p_0 p_1 \dots p_k$ (chosen as a product of $k+1$ small/60-bit primes $\rightarrow$ each fits in a single register), each coefficient c_i is uniquely represented by its $k+1$ residues via the Chinese Remainder Theorem (CRT).

The coefficient, RNS or CRT form of a is then given as:

$$\begin{pmatrix} c_0 \bmod p_0 & c_1 \bmod p_0 & \dots & c_{N-1} \bmod p_0 \\ c_0 \bmod p_1 & c_1 \bmod p_1 & \dots & c_{N-1} \bmod p_1 \\ \vdots & \vdots & \ddots & \vdots \\ c_0 \bmod p_k & c_1 \bmod p_k & \dots & c_{N-1} \bmod p_k \end{pmatrix} \tag{2.3}$$

Clearly, the i-th column of (2.3) is a CRT problem for c_i, so converting between this and the canonical form $\sum_{i=0}^{N-1} c_i X^i$ or vector form $(c_0, c_1, \dots, c_{N-1})$ is easy.

Each row is a smaller polynomial (mod p_i) called a tower, residue or limb

- Addition is highly parallelizable in this format, just add the columns
- Multiplication is still a convolution $\rightarrow \mathcal{O}(n^2)$
- Using Number Theoretic Transform (NTT) $\rightarrow \mathcal{O}(n \log n)$ (this is `DCRTPoly`)

$$\begin{pmatrix} \mathsf{NTT}(a \bmod p_0) \\ \mathsf{NTT}(a \bmod p_1) \\ \vdots \\ \mathsf{NTT}(a \bmod p_k) \end{pmatrix} = \begin{pmatrix} \hat{a}_{0,0} & \hat{a}_{0,1} & \dots & \hat{a}_{0,N-1} \\ \hat{a}_{1,0} & \hat{a}_{1,1} & \dots & \hat{a}_{1,N-1} \\ \vdots & \vdots & \ddots & \vdots \\ \hat{a}_{k,0} & \hat{a}_{k,1} & \dots & \hat{a}_{k,N-1} \end{pmatrix} \tag{2.4}$$

2.2.2 Gadget Decomposition

- Many FHE applications use *gadget decomposition* to limit noise growth

  - RGSW [CGGI18]
  - key switching [**keyswitching**]
  - relinearization [Bra12; BGV11]
- Maybe: Describe problem with (naive) HE multiplication
- Gadget property/vector

Definition 2.2. Gadget property. Let $R_q = \mathbb{Z}_q[X]/(X^N + 1)$ and let $\ell \in \mathbb{N}$. A **gadget vector** is any $g \in R_q^\ell$ for which there exists a decomposition mapping $g^{-1}(x)$ satisfying the **gadget property** (2.10).

$$\langle g^{-1}(x), g \rangle = x + \epsilon \pmod q \tag{2.5}$$

- **Note:** gadget property $\implies$ x is reconstructible **with addition**
- Explanation: encodes the message in base B digits
- Standard choice is $g = (1, 1/B, \ldots, 1/B^\ell)$, giving decomposition function:

$$g^{-1}(x) = (x_0, x_1, \cdots, x_\ell), \text{ where } x_i \text{ is the } i\text{-th digit of } x \text{ in base } B \tag{2.6}$$

- Generalization when x is a vector: gadget matrix $I_n \otimes G$

2.2.3 Key Switching

- Explain key switching
- Uses gadgets, and is implemented in OpenFHE

Brakerski-Vaikuntanathan

- Uses gadget decomposition with fixed radix

Gentry-Halevi-Smart

- Improves

Hybrid

- Combination of BV and GHS

2.2.4 RNS Gadget

- Using the standard gadget decomposition with $g = (1, 1/B, \ldots, 1/B^\ell)$ doesn't make sense in the RNS evaluation space

- We would need to convert back from DCRT to Coefficient and then canonical form.
- However, the (2.4) format already does some kind of decomposition for us, as CRT *is* a form of gadget.

2.3 TFHE (RGSW)

The GSW ciphertext defined in [GSW13], now in the ring R_q. It works by changing bases. Given a base B and decomposition parameter ℓ, such that the message μ can be expressed with ℓ digits in base B, define the *gadget vector* as $g = (1, B, \ldots, B^{\ell-1})^\intercal \bmod t$ and define the *gadget matrix* G as (2.7).

$$G = I_2 \otimes g = \begin{bmatrix} g & 0 \\ 0 & g \end{bmatrix} \tag{2.7}$$

Then an RGSW ciphertext C encrypting a message $\mu \in [0, N)$ is defined by (2.8), where each row of Z is an Ring Learning With Errors (RLWE) encryption of 0 under the secret key s (effectively rows of public keys), μ is the message and G is the gadget matrix.

$$C = Z + \mu \cdot G \tag{2.8}$$

While (2.8) is sufficient to construct $\mathsf{RGSW}_s(\mu)$, we can speed up the process by directly encrypting each row. Note RGSW can be viewed as a flat vector of 2ℓ RLWE ciphertexts where the first ℓ modify a (we call these the *top* rows) and the last ℓ modify b (the *bot* rows).

Corollary 2.3. *For an RGSW ciphertext C encrypting a message μ under s, the $(\ell + i)$-th row $(0 \leq i < \ell)$ is equivalent to:*

$$C_{\ell+i} = \mathsf{RLWE}_s(\mu B^i / \Delta) \tag{2.9}$$

Proof. Follows from the definitions (2.8) and (2.7).

$$\begin{aligned} Z_i + \mu \cdot G_i &= \mathsf{RLWE}_s(0) + \mu \cdot (0, g_i) \\ &= (a, a \cdot s + \epsilon) + (0, \mu \cdot B^i) \\ &= (a, a \cdot s + \mu B^i + \epsilon) \\ &= \mathsf{RLWE}_s(\mu B^i / \Delta) \end{aligned}$$

□

The same approach shows the first ℓ rows can be constructed as an RLWE encryption $c_i = \mathsf{RLWE}_s(-s\mu B^i / \Delta)$ for $0 \leq i < \ell$ (Theorem 2.7). However, calculating $s \cdot \mu$ is not trivial in practice, and it's faster to simply modify the a component of a fresh $\mathsf{RLWE}_s(0)$ encryption. Algorithm 2.1 creates $\mathsf{RGSW}_s(\mu)$ in $\mathcal{O}(\ell)$ steps (but can only be performed on the client-side as it requires s).

Algorithm 2.1 Client-side RGSW generation

Input: Polynomial $\mu = \sum_j b_j X^j, \quad b_j \in [0, N)$
Input: Decomposition parameter ℓ
Output: $\mathsf{RGSW}_s(\mu)$
Construct a $2\ell \times 1$ vector C
for $i = 0,\ i < \ell$ **do**
    Let $\hat{\mu} = \mu B^i / \Delta = \frac{B^i}{\Delta}(\sum_j b_j X^j)$ note: coeff. $B^i \cdot b_j \bmod q$
    $\mathbf{C}_i = \mathsf{RLWE}_s(\hat{\mu})$
    $\mathbf{C}_{i+\ell} = \mathsf{RLWE}_s(0) + (0, \hat{\mu})$
end for
return $\{\mathbf{C}_k\}_{k=0}^{2\ell-1}$

2.3.1 Parameter Choice

The choice of basis B and decomposition parameter ℓ is one and the same. For a given basis, the message polynomial μ is rewritten into $\log_B(\mu)$ base-B digits. Therefore $\ell \geq \log_B(\mu)$ (otherwise the RGSW does not encode all the digits) and $\ell \leq \log_B(\mu)$ (otherwise more B-digits would always be 0). The value of ℓ is squeezed and must equal the number of base-B digits that encodes all μ values allowed by the scheme.

For BGV-RNS with $q = \prod_{i=0}^{k-1} q_i$ and 60-bit primes q_i, this gives $\ell = \lceil \log_B q \rceil = \sum_i \lceil \log_B q_i \rceil$. Even with a large base such as $B = 2^{30}$, ℓ is on the order of $2k$, growing linearly with the multiplicative depth.

2.3.2 External Product (textbook)

First defined in [CGGI18]. The external product (2.11) is defined between an RLWE and a RGSW ciphertext, and outputs an RLWE ciphertext without using homomorphic multiplication (and thus relinearization). The external product uses a *decomposition matrix* G^{-1} that has the *gadget property* (2.10), which effectively "undoes" the effect of the gadget matrix G (with some error $\leq B/2$, assuming x has binary elements).

$$G^{-1}(x) \cdot G = x + e \pmod q \tag{2.10}$$

$$x \boxdot Y = G^{-1}(x) \cdot Y \tag{2.11}$$

2.3.3 Internal Product (textbook)

The *internal product* (2.12) multiplies two RGSW ciphertexts, producing a third RGSW ciphertext. Each row of B is itself an RLWE ciphertext (Theorem 2.3), so

the internal product is 2ℓ external products, one per row, repacked as an RGSW.

$$A \boxtimes B = \begin{bmatrix} A \boxdot b_0 \\ A \boxdot b_1 \\ \vdots \\ A \boxdot b_{2\ell-1} \end{bmatrix} \tag{2.12}$$

2.3.4 The Hybrid-Keyswitch RNS Gadget

The textbook gadget $g = (1, B, \ldots, B^{\ell-1})$ from subsection 2.3.1 pays a steep price in BGV-RNS: $\ell \approx 2k$ rows, each a full RLWE ciphertext. The *hybrid keyswitch* construction (used by OpenFHE for relinearization) replaces this with an RNS-native gadget of length dnum where typically $\mathsf{dnum} \in \{2, 3, 4\}$, independent of B. We adapt it to RGSW.

The idea: instead of decomposing each coefficient into base-B digits, we decompose the *tower index* i into a small number of partitions, and use the QP-basis identity from **??** to cancel the gadget factor.

Q-partitions

Partition the k RNS towers of Q into dnum contiguous groups of $\alpha = \lceil k/\mathsf{dnum} \rceil$ towers each (the last group may be shorter):

$$\mathsf{part}_j = \{i \in [0, k) : j\alpha \leq i < (j+1)\alpha\}, \qquad j \in [0, \mathsf{dnum}). \tag{2.13}$$

Let $Q_j = \prod_{i \in \mathsf{part}_j} q_i$ so that $Q = \prod_{j=0}^{\mathsf{dnum}-1} Q_j$. In OpenFHE, dnum is set via `CCParams::SetNumLargeDigits`, and α is exposed as `numPerPartQ`.

The partition-indicator gadget element

For each partition j, define the QP-element $g_j \in R_{QP}$ tower-by-tower:

$$g_j[i] = \begin{cases} [P]_{q_i} \pmod{q_i} & i \in \mathsf{part}_j \quad (Q\text{-side}) \\ 0 & i \in [0, k) \setminus \mathsf{part}_j \quad (Q\text{-side}) \\ 0 & i \in P\text{-side} \end{cases} \tag{2.14}$$

That is: g_j is the QP element holding the value $[P]_{q_i}$ in those Q-towers that belong to partition j, and zero everywhere else. The constants $[P]_{q_i}$ are precomputed in OpenFHE as `cryptoParams->GetPModq()`.

Remark 2.4. The textbook gadget $g_i = B^i$ used a single integer per row; the hybrid gadget g_j uses an entire QP-element per row, but the element is sparse (zero outside one Q-partition) and its non-zero entries are determined by the auxiliary modulus P.

Digit decomposition

Given $c \in R_Q$, define the j-th digit $\mathsf{Decomp}_j(c) \in R_{QP}$ as the basis-extension of c's partition-j slice into all of QP. Concretely, take the residues $\{c \bmod q_i : i \in \mathsf{part}_j\}$

as a polynomial in R_{Q_j}, then $\mathsf{ApproxSwitchCRTBasis}$ from Q_j to QP. This gives a vector of dnum elements in R_{QP}, one per partition.

In OpenFHE this routine is `KeySwitchHYBRID::EvalKeySwitchPrecomputeCore`.

Lemma 2.5. Gadget identity *For any $c \in R_Q$,*

$$\sum_{j=0}^{\mathsf{dnum}-1} \mathsf{Decomp}_j(c) \cdot g_j \;\equiv\; P \cdot c \pmod{Q}, \qquad \equiv\; 0 \pmod{P}. \tag{2.15}$$

Proof. Examine each tower of QP separately.

Q-side, tower $i \in \mathsf{part}_j$: For the partition $j' = j$ that contains i, $\mathsf{Decomp}_j(c)[i] = c[i]$ exactly — the partition-j slice contains tower i natively, no basis extension needed. And $g_j[i] = [P]_{q_i}$, so the contribution is $c[i] \cdot [P]_{q_i} \equiv P \cdot c[i] \pmod{q_i}$. For all other partitions $j' \neq j$, $g_{j'}[i] = 0$, contributing nothing. Total: $P \cdot c[i] \pmod{q_i}$.

P-side, tower i: $g_j[i] = 0$ for all j, so the sum is 0. □

Compare (2.15) with the textbook $\sum_i u_i \cdot B^i = c$: the two play the same role, but the hybrid form picks up a free factor of P on the Q-side that we will cancel with $\mathsf{ApproxModDown}$ (**??**).

V2 RGSW format

Define $\mathsf{RGSW}_s(m)$ as two arrays of dnum pairs in R_{QP}^2 each:

$$b_j^{\mathsf{top}} + a_j^{\mathsf{top}} \cdot s \;\approx\; m \cdot s \cdot P \cdot g_j \pmod{QP} \tag{2.16}$$

$$b_j^{\mathsf{bot}} + a_j^{\mathsf{bot}} \cdot s \;\approx\; m \cdot P \cdot g_j \pmod{QP} \tag{2.17}$$

for $j \in [0, \mathsf{dnum})$. The factor P on the right-hand side comes from g_j (whose non-zero entries are $[P]_{q_i}$); we factor it out as a separate P for clarity in the derivations below. This is implemented in `EncryptRGSW` (Algorithm 2.2).

The structure mirrors `KeySwitchHYBRID::KeySwitchGenInternal` exactly, with the message m playing the role of the "old" secret key in the keyswitch.

V2 External product, derived

Given an RLWE ciphertext $x = (a_x, b_x) \in R_Q^2$ encrypting m_x (so $b_x + a_x s \approx m_x \pmod{q}$) and an RGSW ciphertext $Y = \mathsf{RGSW}_s(m_y)$ in V2 format, compute:

1. $\mathsf{vDig}[j] = \mathsf{Decomp}_j(b_x)$, $\quad \mathsf{uDig}[j] = \mathsf{Decomp}_j(a_x)$ for $j \in [0, \mathsf{dnum})$.

Algorithm 2.2 V2 RGSW encryption (server-side, mirrors hybrid keyswitch)

```
Input: secret key s ∈ R_Q; message m ∈ R_t; partition count dnum
Output: RGSW_s(m) in V2 format
Encode m ∈ R_t as m_Q ∈ R_Q, then extend s → s_QP ∈ R_QP
for j = 0, ..., dnum − 1 do
    for side ∈ {top, bot} do
        Sample a ← U(R_QP) and e ← DGG(R_QP)
        Let payload_Q = m_Q · s (top) or m_Q (bot)
        for tower i ∈ [0, sizeQ + sizeP) do
            if i ∈ part_j (Q-side) then
                b[i] ← −a[i] · s_QP[i] + e[i] + [P]_{q_i} · payload_Q[i]
            else
                b[i] ← −a[i] · s_QP[i] + e[i]
            end if
        end for
        Store (a, b) as (a_j^side, b_j^side)
    end for
end for
return {(a_j^top, b_j^top), (a_j^bot, b_j^bot)}_{j=0}^{dnum−1}
```

2. Inner products in R_{QP}:

$$r_0 = \sum_j \mathsf{uDig}[j] \cdot b_j^{\mathsf{top}} + \mathsf{vDig}[j] \cdot b_j^{\mathsf{bot}} \tag{2.18}$$

$$r_1 = \sum_j \mathsf{uDig}[j] \cdot a_j^{\mathsf{top}} + \mathsf{vDig}[j] \cdot a_j^{\mathsf{bot}} \tag{2.19}$$

3. Output $(\mathsf{ApproxModDown}(r_1), \mathsf{ApproxModDown}(r_0)) \in R_Q^2$.

Lemma 2.6. V2 external product correctness $\mathsf{ApproxModDown}(r_0) + \mathsf{ApproxModDown}(r_1) \cdot s \approx m_x \cdot m_y \pmod q$.

Proof. Compute $r_0 + r_1 \cdot s$ in R_{QP} using (2.16)–(2.17):

$$r_0 + r_1 s = \sum_j \mathsf{uDig}[j] \cdot \left(b_j^{\mathsf{top}} + a_j^{\mathsf{top}} s\right) + \mathsf{vDig}[j] \cdot \left(b_j^{\mathsf{bot}} + a_j^{\mathsf{bot}} s\right) \tag{2.20}$$

$$\approx \sum_j \mathsf{uDig}[j] \cdot m_y \cdot s \cdot P \cdot g_j + \sum_j \mathsf{vDig}[j] \cdot m_y \cdot P \cdot g_j \tag{2.21}$$

$$= m_y \cdot s \cdot P \cdot \underbrace{\sum_j \mathsf{uDig}[j] \cdot g_j}_{\equiv P \cdot a_x} + m_y \cdot P \cdot \underbrace{\sum_j \mathsf{vDig}[j] \cdot g_j}_{\equiv P \cdot b_x} \quad \text{(by Theorem 2.5)}$$

$$\equiv m_y \cdot P^2 \cdot (a_x s + b_x) \pmod Q \tag{2.22}$$

$$\approx m_y \cdot P^2 \cdot m_x \pmod Q. \tag{2.23}$$

On the P-side, both gadget sums are 0 (Theorem 2.5), so $r_0 + r_1 s \equiv 0 \pmod{P}$. Applying **??** to $\mathsf{ApproxModDown}$:

$$\mathsf{ApproxModDown}(r_0)+\mathsf{ApproxModDown}(r_1)\cdot s \approx \frac{m_y \cdot P^2 \cdot m_x}{P} = m_y \cdot P \cdot m_x \cdot \frac{1}{P} \equiv m_x m_y \pmod{q}, \tag{2.24}$$

since the second $\mathsf{ApproxModDown}$ application strips the remaining P factor. □

Implementation note. Theorem 2.6 corresponds line-by-line to `EvalExternalProduct` in `src/core/context.cpp`. Step 1 is `EvalKeySwitchPrecomputeCore` (lines 99–101), step 2 is the inner loop (lines 114–129), step 3 is `ApproxModDownToQ` (lines 132–133). The total cost is *2 ModUps +* $2\mathsf{dnum} \cdot \mathsf{sizeQP}$ *pointwise multiplies + 2 ApproxModDowns* — versus $2\ell \approx 4k$ full RLWE rows in the textbook version.

V2 Internal product, derived

For each row $(a_j^{\mathsf{side}}, b_j^{\mathsf{side}})$ of $B = \mathsf{RGSW}_s(m_B)$, observe that it is an RLWE-shaped pair in R_{QP} encrypting $m_B \cdot P \cdot g_j$ (or $m_B \cdot s \cdot P \cdot g_j$ for top rows). To produce the corresponding row of $\mathsf{RGSW}_s(m_A m_B)$, we apply the external product action of A and re-fit into the V2 format:

1. Project the row from R_{QP} to R_Q via $\mathsf{ApproxModDown}$, cancelling the gadget's P factor (**??**). The result is an RLWE ciphertext in R_Q encrypting $m_B \cdot g_j$ (bot) or $m_B \cdot g_j \cdot s$ (top), where g_j is interpreted as the Q-only indicator (value 1 on partition j, 0 elsewhere).
2. Apply the V2 external product of A to this row. The result is an RLWE ciphertext in R_Q encrypting $m_A \cdot m_B \cdot g_j$ (or with an extra $\cdot s$ for top).
3. Lift back to R_{QP} by multiplying each Q-tower by $[P]_{q_i}$ and zeroing the P-side. This restores the V2 format (2.16)–(2.17) with $m \mapsto m_A m_B$.

This is implemented in `EvalInternalProduct` (`context.cpp:287-381`). The cost is $2\mathsf{dnum}$ external products plus the round-trip overhead, i.e. $\mathcal{O}(\mathsf{dnum}^2)$ ring multiplications — versus the textbook $\mathcal{O}(\ell^2) \approx \mathcal{O}(k^2)$.

Per-level alignment

When x has been mod-reduced to $Q_\ell \subset Q$ (some subset of the towers), the digit decomposition automatically returns fewer digits ($\mathsf{numPartQ}_\ell \leq \mathsf{dnum}$), and the inner loop indexes into the full-Q RGSW with offset $\delta = \mathsf{sizeQ} - \mathsf{sizeQ}_\ell$ to skip dropped towers. This is the same trick OpenFHE's `EvalFastKeySwitchCoreExt` uses, and gives mod-reduced $x \boxdot Y$ for free without re-encrypting the RGSW at each level.

V1 vs V2: comparison

	V1 (per-tower BV)	**V2 (RNS digit)**
Gadget length	$\ell = \sum_i \lceil \log_B q_i \rceil \approx 2k$	dnum, typically 2–4
Gadget element	integer scalar B^i	QP-element g_j with $[P]_{q_i}$
Per-row primitive	full RLWE ciphertext	raw `DCRTPoly` pair in QP
Decomposition	per-coefficient base-B extraction	one `ApproxSwitchCRTBasis` per side
Gadget property	$\sum_i u_i B^i = c$	$\sum_j \mathsf{uDig}[j] \cdot g_j \equiv P \cdot c \pmod{Q}$
Auxiliary cancellation	none	$\mathsf{ApproxModDown}$ removes P
External product cost	2ℓ `ScalarMult+EvalAdd`	2 ModUps + $2\mathsf{dnum}$ inner-product st
Per-level alignment	manual	automatic via δ-skip
Internal product cost	$\mathcal{O}(\ell^2)$	$\mathcal{O}(\mathsf{dnum}^2)$

Table 2.1: V1 (textbook) vs V2 (hybrid-keyswitch) RGSW. The V2 form reduces the inner loop length by $\ell/\mathsf{dnum} \approx k/2$, replaces ciphertext-level operations with `DCRTPoly`-level operations, and reuses OpenFHE's hybrid-keyswitch machinery for digit decomposition and modulus reduction.

The novel angle: in V2 the gadget identity (2.15) carries an extra factor of P, exactly the auxiliary modulus that $\mathsf{ApproxModDown}$ cancels. The external product therefore fuses three operations into a single coherent pass: digit decomposition (ModUp), inner product against the gadget rows, and removal of the auxiliary modulus (ModDown) — the same machinery as relinearization, with dnum as the single shared parameter (`SetNumLargeDigits`).

2.3.5 RLWE-to-RGSW Expansion from ORAM

As established in section 2.3, RGSW ciphertexts are enticing because the external product allows us to perform homomorphic multiplication without relinearization. However, the size of RGSW makes the communication costs very high in a communication protocol. We wish to strike a balance between the cheap communication cost of packed RLWE and the fast multiplication of RGSW. The solution is to send RLWE ciphertexts from the client, and *expand* them homomorphically on the server.

Solution from [CCR19]. **HomExpand:** Expands an RLWE ciphertext into an RGSW ciphertext.

This expansion is described in Algorithm 4 of [CCR19], which builds on Algorithms 3 and 8 from the same paper. The algorithms are listed below (in slightly different notation) as Algorithm 2.3, Algorithm 2.4, and **??**, respectively.

Algorithm 2.3 HomExpand

Input: $\mathbf{c}_k = \mathrm{RLWE}(\sum_{i=0}^{n-1} \frac{b_i}{nB^{k+1}}),\ b_i \in \{0,1\},\ 0 \le k < \ell$
Input: $A = \mathrm{RGSW}(-s)$
Output: $\mathbf{C}_i = \mathrm{RGSW}(b_i),\ 0 \le i < n$
for each k **do**
    $\mathbf{c'}_k := \mathsf{expandRlwe}(c_k)$
end for
for $i = 0,\ i < n$ **do**
    Initialize $\mathbf{C}_i$ as an empty $2\ell \times 2$ matrix of polynomials
    for $k = 0; k < \ell$ **do**
        $\mathbf{C}_i[k] = A \boxdot \mathbf{c'}_k[i]$
        $\mathbf{C}_i[k+\ell] = \mathbf{c'}_k[i]$
    end for
end for
return $\{\mathbf{C}_j\}_{j=0}^{n-1}$

Algorithm 2.3 takes as input ℓ RLWE ciphertexts, which in sPAR correspond to the ℓ encrypted bits of the index a_j. It also takes an RGSW ciphertext of the encrypted secret key (which key, automorphism? Sent by the client or generated by server?).

It begins by running a subroutine expandRlwe (Algorithm 2.4) which converts each of the ℓ RLWE ciphertexts into a matrix, like a *partial* RGSW. Then these partial RGSW's are combined into a single (complete) RGSW.

This subroutine converts an RLWE ciphertext with ℓ slots into ℓ ciphertexts, each encoding a unique slot of the input. Subs is a function that... (EvalAutomorphism!)

Algorithm 2.4 expandRlwe

Input: $\mathbf{c} = \mathrm{RLWE}(\sum_{i=0}^{n-1} b_i X^i),\ b_i \in \{0,1\}$
Output: $\mathbf{c}_i = \mathrm{RLWE}(n \cdot b_i),\ 0 \leq i < n$
$\mathbf{c}_0 := \mathbf{c}$
for $i = 0,\ i \leq \log n$ **do**
    $k = n/2^{i-1} + 1$
    for $b = 0; b < 2^i$ **do**
        $\mathbf{c}_{2b} = c_b + \mathrm{Subs}(c_b, k)$
        $\mathbf{c}_{2b+1} = c_b - \mathrm{Subs}(c_b, k)$
    end for
end for
return $\{\mathbf{c}_j\}_{j=0}^{n-1}$

Their method is fast for TFHE, but for our use — case where $n << N$ — it's faster to use the method described in [HS18] to repeatedly "rotate" the ciphertext ℓ times as in Algorithm 2.5. The FastRotate operation is supported by the OpenFHE as EvalFastRotate.

Note that the components being scaled by n is a side-effect of their implementation, and not required when using FastRotate, so the output is a list of $c_i = \mathsf{RLWE}(b_i)$.

Algorithm 2.5 expandRlwe

Input: $\mathbf{c} = \mathrm{RLWE}(\sum_{i=0}^{n-1} b_i X^i),\ b_i \in \{0,1\}$
Output: $\mathbf{c}_i = \mathrm{RLWE}(b_i),\ 0 \leq i < n$
$\mathbf{c}_0 := \mathbf{c}$
for $i = 0,\ i \leq \log n$ **do**
    do something...
end for
return $\{\mathbf{c}_j\}_{j=0}^{n-1}$

2.3.6 Homomorphic Field Trace

New paper!

2.4 Implementation Details

A ciphertext is:

$$(c_o, c_1) \implies c_0 + c_1 \cdot s \equiv m + tv \pmod{Q} \tag{2.25}$$

In BGV:

$$(c_0, c_0 \cdot s + m + \epsilon) \implies c_0 + c_0 \cdot s + m + \epsilon = c_0(s \tag{2.26}$$

2.4.1 RGSW Encryption (textbook form)

In practice, it would be convenient to construct each row directly, rather than first constructing both matrices in (2.8) and combining them.

Recall that an RLWE ciphertext is a vector $\mathsf{RLWE}(\mu) = (a, b)$ where $b = a \cdot s + \mu + \epsilon$ *(since $\Delta = 1$ in BGV)*. We can view RGSW as a flat vector of 2ℓ RLWE ciphertexts, or a $2\ell \times 2$ matrix with an a column and b column. In the latter representation, the first ℓ rows of C are expressed as:

$$C_i = (a,\ b_{\mu=0}) + \mu \cdot (1/B^i,\ 0) = (a + \mu/B^i,\ a \cdot s + \epsilon)$$

As each row of an RGSW ciphertext is a valid RLWE ciphertext, we can find a new message $\hat{\mu}_i$ such that $\mathsf{RLWE}(\hat{\mu}_i) = (\hat{a}, \hat{a} \cdot s + \Delta \cdot \hat{\mu}_i + \epsilon) = C_i$. Starting with the top half, let $\hat{a} = a + \mu \cdot B^i$ so that the first component matches.

$$\begin{aligned} b &= (a + \mu \cdot B^i) \cdot s + \Delta \cdot \hat{\mu}_i + \epsilon && (2.27)\\ &= a \cdot s + s \cdot \mu \cdot B^i + \Delta \cdot \hat{\mu}_i + \epsilon && (2.28)\\ &= a \cdot s + (s\mu B^i + \Delta \hat{\mu}_i) + \epsilon && (2.29) \end{aligned}$$

The middle term must equal 0 to match C_i, which leads us to an expression (2.32) for $\hat{\mu}_i$.

$$\begin{aligned} 0 &= s \cdot \mu \cdot B^i + \Delta \cdot \hat{\mu}_i && (2.30)\\ \hat{\mu}_i &= -s \cdot \mu \cdot B^i \cdot 1/\Delta && (2.31)\\ \hat{\mu}_i &= -s \cdot \mu \cdot B^i && (2.32) \end{aligned}$$

Therefore we can construct row i (0-indexed) of $\mathsf{RGSW}_s(\mu)$ as $\mathsf{RLWE}_s(-s \cdot \mu \cdot B^i)$ for $0 \leq i < \ell$. Further, the RLWE plaintext μ is mod t so we can also use $\mathsf{RLWE}_s(t - s\mu B^i)$.

$$\begin{aligned}
\mathsf{RLWE}_s(-s\mu B^i) &= (a, a \cdot s - s\mu B^i + \epsilon) \pmod{t} \\
\mathsf{RLWE}_s(t - s\mu B^i) &= (a, a \cdot s + (t - s\mu B^i) + \epsilon) \pmod{t} \\
\mathsf{RLWE}_s(-s\mu B^i) &\equiv \mathsf{RLWE}_s(t - s\mu B^i) \pmod{t}
\end{aligned}$$

For the bottom ℓ rows, we can skip Z entirely since, as Theorem 2.3 says:

$$\mathsf{RLWE}_s(0) + (0, \mu B^i) = (a,\ a \cdot s + \mu B^i + \epsilon) = \mathsf{RLWE}_s(\mu B^i)$$

Lemma 2.7. *The i-th row C_i of $\mathsf{RGSW}_s(\mu)$ is:*

$$C_i = \mathsf{RLWE}_s(-s\mu B^i/\Delta) \tag{2.33}$$

Proof. Let $\hat{a} = a + \mu B^i$, and consider $\mathsf{RLWE}_s(\hat{\mu})$.

$$\begin{aligned}
\mathsf{RLWE}_s(\hat{\mu}) &= (\hat{a}, \hat{a} \cdot s + \Delta\hat{\mu} + \epsilon) \\
&= (a + \mu B^i, (a + \mu B^i) \cdot s + \Delta\hat{\mu} + \epsilon) \\
&= (a + \mu B^i, a \cdot s + \mu B^i \cdot s + \Delta\hat{\mu} + \epsilon)
\end{aligned}$$

Assume $C_i = \mathsf{RLWE}_s(\hat{\mu})$, then:

$$\begin{aligned}
s\mu B^i + \Delta\hat{\mu} &= 0 \\
\therefore \hat{\mu} &= -s\mu B^i/\Delta
\end{aligned}$$

□

2.4.2 V2 RGSW Encryption

The hybrid-keyswitch (V2) RGSW format defined in section 2.3.4 is constructed directly from the secret key s via Algorithm 2.2. The full derivation, including the gadget identity (Theorem 2.5) and external product correctness (Theorem 2.6), is given in subsection 2.3.4. The implementation in `src/core/context.cpp` (`EncryptRGSW`, `EvalExternalProduct`, `EvalInternalProduct`) follows that section line-by-line.

2.5 (Somewhat) Practical Anonymous Routing

The main protocol.

2.5.1 HomPlacing

Algorithm 1 shows the idea of Algorithm 2.

- From NIAR: sorting of random numbers = shuffling.
- Evaluate tree, each bit determines if the data is placed left or right. One level per bit means there are 2^n buckets in the end, each one representing a bit string i.e. 000, 001, 010, 011, 100, 101, 110, 111 for 3 bits.
- We can encode the bits into RGSW form.

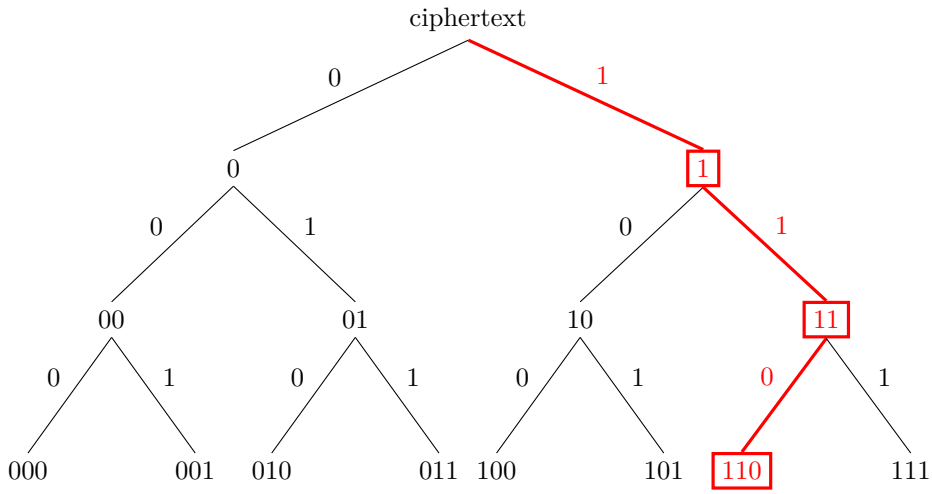

Figure 2.1: HomPlacing tree with a 3-bit index a. At level i, the i-th MSB of the index a determines homomorphically wether the ciphertext is placed in the left (0) or right (1) bucket. This process is repeated $\log n = 3$ times until the ciphertext is placed in the a-th bucket, without the placer ever knowing which bucket that is. The highlighted path shows the route taken for the index $a = 6 = [110]_2$: the MSB places the ciphertext in the right bucket, and so on. Again, the server doesn't know a, only the *encryption* of the bits $[a]_2$.

Algorithm 2.6 Homomorphic placing HomPlacing for one slot

1: **Input:** An encrypted value V_r and $\log n$ bits $\{c_0, \ldots, c_{L-1}\}$, where $L = \log n$
2: **Output:** A vector $\mathbf{l}$ consisting of n values of leaves: $z_0, \ldots, z_{n-1}$, where $\mathbf{l}[i] = z_i$ for $i \in \{0, \ldots, n-1\}$
3: $\mathbf{l} := \mathbf{0}$
4: Initialize root: $b_0 = V_r$
5: Initialize the node values: $b_i := 0$ for $i \in \{1, \ldots, 2n-2\}$
6: **for** $i \leftarrow 0, \ldots, L-1$ **do**
7: **for** $j \leftarrow 0, \ldots, 2^i - 1$ **do**
8: $r := b[2^i - 1 + j]$
9: $b_{2^{i+1}+2j} := r \boxdot c_i$
10: $b_{2^{i+1}+2j-1} := r - r \boxdot c_i$
11: **end for**
12: **end for**
13: **for** $i \leftarrow 0, \ldots, n-1$ **do**
14: $z_i := b_{2^{l+1}-1+i}$
15: $\mathbf{l}[i] = z_i$
16: **end for**
17: **return** $\mathbf{l}$

Listing 1 Algorithm 1 from sPAR implemented in C++ with OpenFHE

```
using namespace lbcrypto;

template <typename T> using Vec = std::vector<Ciphertext<T>>;
template <typename Element> using CC = CryptoContext<Element>;
template <typename Element> using CT = Ciphertext<Element>;
template <typename Element> using CTVec = Vec<Element>;

template <typename Element>
CTVec<Element> HomPlacingNoExt(
    const CC<Element>&      cc,
    const CT<Element>&      value,
    const CTVec<Element>    c
)
{
    // Levels in tree L = log(n)
    const uint32_t L = c.size();
    const uint32_t n = 1u << L;

    // Initialize b = { V_r, 0, 0, ..., 0 }
    CTVec<Element> b(2*n - 1);
    b[0] = value;

    for(uint32_t i = 0; i < L; i++) {
        const CT<Element>& bit = c[i];
        // Iterate over all nodes in i-th level
        for(uint32_t j = 0; j < (1u << i); j++) {
            const uint32_t idx_right = (1u << (i + 1)) + 2*j;
            const uint32_t idx_left = idx_right - 1;
            const auto& parent = b[(1u << i) - 1 + j];
            b[idx_right] = cc->EvalMult(parent, bit);
            b[idx_left] = cc->EvalSub(parent, b[idx_right]);
        }
    }

    // Output last n nodes
    CTVec<Element> leaves(n);
    for(uint32_t i = 0; i < n; i++)
        leaves[i] = b[n - 1 + i];

    return leaves;
}
```

2.6 Multi-Party Homomorphic Encryption

TODO

Chapter 3
Methodology

3.1 Variables

- **Plaintext modulus** t - Coefficients of plaintext polynomial are taken mod t
- **Ciphertext modulus** Q - Decomposed into $Q = q_0 q_1 \dots q_L$

3.2 Notes

In Brakerski's scheme, similar to previous FHE schemes based on LWE, the ciphertext $\vec{c}$ and secret key $\vec{s}$ are n-dimensional vectors whose dot product $\langle \vec{c}, \vec{s} \rangle \approx \mu$ equals the message μ, up to some small "error" that is removed by rounding.

3.3 Residue Number System

In RNS, each ciphertext is already decomposed mod $Q_L = q_0 q_1 \dots q_L$. If, then, we select a gadget consisting of $(q_0, q_1, \dots, q_L)$, we get the decomposition "for free".

Chapter 4
Results and Discussion

Chapter 5
Conclusion and Future Work

References

[BEHZ16] J.-C. Bajard, J. Eynard, et al., *A full RNS variant of FV like somewhat homomorphic encryption schemes*, Cryptology ePrint Archive, Paper 2016/510, 2016. [Online]. Available: https://eprint.iacr.org/2016/510.

[BGV11] Z. Brakerski, C. Gentry, and V. Vaikuntanathan, *Fully homomorphic encryption without bootstrapping*, Cryptology ePrint Archive, Paper 2011/277, 2011. [Online]. Available: https://eprint.iacr.org/2011/277.

[Bra12] Z. Brakerski, *Fully homomorphic encryption without modulus switching from classical GapSVP*, Cryptology ePrint Archive, Paper 2012/078, 2012. [Online]. Available: https://eprint.iacr.org/2012/078.

[CCR19] H. Chen, I. Chillotti, and L. Ren, "Onion ring oram: Efficient constant bandwidth oblivious ram from (leveled) tfhe", in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '19, London, United Kingdom: Association for Computing Machinery, 2019, pp. 345–360. [Online]. Available: https://doi.org/10.1145/3319535.3354226.

[CGGI18] I. Chillotti, N. Gama, et al., *TFHE: Fast fully homomorphic encryption over the torus*, Cryptology ePrint Archive, Paper 2018/421, 2018. [Online]. Available: https://eprint.iacr.org/2018/421.

[Gen09] C. Gentry, "A fully homomorphic encryption scheme", Ph.D. dissertation, Stanford University, 2009. [Online]. Available: https://crypto.stanford.edu/craig.

[GSW13] C. Gentry, A. Sahai, and B. Waters, *Homomorphic encryption from learning with errors: Conceptually-simpler, asymptotically-faster, attribute-based*, Cryptology ePrint Archive, Paper 2013/340, 2013. [Online]. Available: https://eprint.iacr.org/2013/340.

[HS18] S. Halevi and V. Shoup, *Faster homomorphic linear transformations in HElib*, Cryptology ePrint Archive, Paper 2018/244, 2018. [Online]. Available: https://eprint.iacr.org/2018/244.

[KPZ21] A. Kim, Y. Polyakov, and V. Zucca, *Revisiting homomorphic encryption schemes for finite fields*, Cryptology ePrint Archive, Paper 2021/204, 2021. [Online]. Available: https://eprint.iacr.org/2021/204.

[RAD78] R. L. Rivest, L. Adleman, and M. L. Dertouzos, "On data banks and privacy homomorphisms", in *Foundations of Secure Computation*, 1978, pp. 169–180.

[SEV+15] Y. Sun, A. Edmundson, et al., "RAPTOR: Routing attacks on privacy in tor", in *24th USENIX Security Symposium (USENIX Security 15)*, Washington, D.C.: USENIX Association, Aug. 2015, pp. 271–286. [Online]. Available: https://www.usenix.org/conference/usenixsecurity15/technical-sessions/presentation/sun.

[SW21] E. Shi and K. Wu, *Non-interactive anonymous router*, Cryptology ePrint Archive, Paper 2021/435, 2021. [Online]. Available: https://eprint.iacr.org/2021/435.

Appendix A
Algebra

A refresher on relevant basic algebra and number theory. Maybe some can be used in the thesis or as an appendix.

A.1 Rings

A polynomial with coefficients in $\mathbb{F}$ is denoted $\mathbb{F}[X]$. The quotient of a ring $\mathbb{F}[X]/S$ is just $\mathbb{F}[X]$ mod S. Most FHE schemes are based on rings (RLWE, RGSW etc.), and in particular we often see the ring $(\mathbb{Z}/q\mathbb{Z})[X]/(f(X))$ for some integer q and some polynomial $f(X)$. Here $(\mathbb{Z}/q\mathbb{Z})[X]$ denotes a polynomial with coefficients in $\mathbb{Z}_q$ i.e. integers mod q, and since we have $\mathbb{Z}_q[X]/(f(X))$ it means that $f(x) \equiv 0$ in this ring. A natural choice is $f(X) = X^n + 1$ for many of these schemes for some variable n, as this allows all polynomials with degree (up to) n. (And is a cyclotomic polynomial $\rightarrow$ has additional structure that can be used to optimize performance.)

If $f(X)$ is irreducible over $\mathbb{F}$, then $\mathbb{F}[X]/(f(X))$ is a field. (And $f(X)$ is a maximum ideal of $\mathbb{F}[X]$.)

A factor group G/H, pronounced *G mod H*, is a factor group where H **is the identity element**. In other words, $\mathbb{F}[X]/(f(X))$ has the (additive) identity $f(X)$ aka. $f(X) \equiv 0$ in the ring.

A.2 Cyclotomic Polynomials

$(X^n + 1)$ is a *cyclotomic polynomial*. They are irreducible over $\mathbb{Z}$ as long as $n = 2^m$, $m \in \mathbb{Z}$ (in other words, n is a power of 2) but are typically not irreducible over $\mathbb{Z}_q$.

Definition A.1. For any positive integer n, the n-th roots of unity are the (complex) solutions to the equation $x^n = 1$, and there are n solutions to the equation.

Let $\zeta_n(x) = e^{2\pi i}$. It follows from Euler's formula $e^{2\pi i} = \sin 2\pi + i \cos 2\pi = 1$ that $\zeta_n^k(x) = \{1, e^{2\pi i}, e^{4\pi i}, \ldots, e^{2k\pi i}\}$ for $0 \leq k < n$ are the n-th roots of unity. At least this is the case over $\mathbb{C}$, but we can define the same problem over different fields.

Geometrically, we can interpret the n-th roots of unity as the points that are evenly spread on the unit circle in the complex plane, starting from 1 on the real axis. (The word *cyclotomic* means *circle-dividing.*)

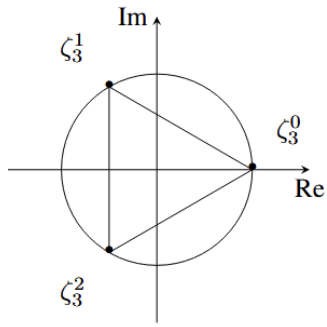

Figure A.1: TODO: Create in tikz or drop

We can define this notion of different fields, and are most interested in defining it over a *finite* $\mathbb{Z}_{p^k}$. As an example, consider $\mathbb{Z}_7 = \{0, 1, 2, 3, 4, 5, 6\}$. The 4-th roots of unity are only 1 and 6, as these are the only values equivalent to 1 when raised to the fourth power. Note that this field has $\phi(n) = \phi(4) = 2$ number of n-th roots for $n = 4$. This is always the case for any finite field and choice of n.

$$1^4 \equiv 1 \mod 7 \tag{A.1}$$

$$6^4 \equiv 1 \mod 7 \tag{A.2}$$

Theorem A.2. *A finite field $\mathbb{Z}_{p^k}$ has a primitive n-th root if and only if $n \mid (p^k - 1)$. In this case, there are $\phi(n)$ roots of unity!*

Definition A.3. An n-th root of unity r is called **primitive** if it is not a d-th root of unity for any integer d smaller than n.

Theorem A.4. *The n-th primitive roots of unity are precisely:*

$$\{\zeta_n^k \,:\, 1 \leq k \leq n-1,\ \gcd(k, n) = 1\} \tag{A.3}$$

Definition A.5. The n-th cyclotomic polynomial $\Phi_n(x)$ has the n-th primitive roots of unity as its roots:

$$\Phi_n(x) = \prod_{\substack{1 \leq k < n \\ \gcd(k,n)=1}} (x - \zeta_n^k) \tag{A.4}$$

Again, we care only about a special class of cyclotomic polynomials that simplify our work. One step in this direction is considering only when n is prime:

$$\Phi_n(x) = x^{n-1} + x^{n-2} + \cdots + 1 = \sum_{t=0}^{n-1} x^t \tag{A.5}$$

Now we consider only finite fields $\mathbb{GF}(p^k)$. Then the cyclotomic polynomials look like:

$$\Phi_n(x) = \Phi_p(x^{n/p}) = \Phi_p(x^{p^{k-1}}) = \sum_{t=0}^{p-1} x^{tp^{k-1}} \tag{A.6}$$

And finally, when $p = 2$ so $n = 2^k$ we get (A.7) where $m = 2^{k-1}$, $n = 2m$,.

$$\Phi_n(x) = x^m + 1 \tag{A.7}$$

Cyclotomic polynomials are monic (implied by the definition) and has $\phi(n)$ linear factors. In addition, $\Phi_n(x)$ divides $x^n - 1$. This implies an important relationship:

$$x^n - 1 = \prod_{d|n} \Phi_d(x) \tag{A.8}$$

The identity (A.8) says that a number is an n-th root of unity iff. it is a d-th primitive root of unity for some natural number d that divides n.

A.3 Galois Group of Cyclotomic Polynomials

Galois theory can be used to answer the question of whether the roots of an arbitrary polynomial f can be expressed in terms of its coefficients. The method for doing so leads to the definition of the *splitting field of a polynomial*, which contains the elementary symmetric polynomials and other polynomials of (subsets of) the roots that can be obtained from them??

Definition A.6. Let f be a polynomial with rational coefficents. The splitting field K of f is the smallest field that contains the roots of f.

K is called the splitting field of f because we can split f into linear factors in K.

Appendix B
Number Theory

Used to speed up calculations on polynomials.

B.1 Chinese Remainder Theorem

Works the exact same way as for integers.

B.2 CRT Polynomial

Used to reduce the size of (the representation of) large polynomials, as q is often much larger than a computer word (32- or more commonly 64-bit words)

B.3 Double CRT Polynomial

Used to perform multiplication on the smaller coefficient-first representation of large polynomials.

B.4 Number Theoretic Transform

The fast fourier transform (FTT) is an algorithm often used to simplify calculations in signals processing etc. by moving a continuous function from the time domain to the frequency domain. The NTT works in a similar fashion; it's essentially a FFT mod q used to speed up multiplication of polynomials.

It is performed on DCRT polynomials.